Echte Daten

Termin 7 · 24.11.2026

FÄLLIG
bis Termin 8 am 01.12.

ZEITBEDARF
ca. 2,5 Stunden

BAUSTEINE IM REPO
t7-fetch-zustaende, t7-api-spielwiese, t7-bibliothek-chartjs

KONTAKT
dhbw@theaisoftwarecompany.com

Heute holt euer Projekt echte Daten aus dem Netz. Der schwierige Teil ist nicht `fetch` – der schwierige Teil sind die Momente, in denen die Antwort ausbleibt.

Teil A · Pflicht

A1 API auswählen und prüfen

Sucht eine öffentliche API, die zu eurem Projekt passt. Beantwortet **vor** der ersten Zeile Code:

1. Braucht sie einen Key? Ist er für den Browser gedacht oder geheim?
2. Wie viele Anfragen pro Stunde sind erlaubt?
3. Erlaubt sie Zugriff von einer anderen Domain (CORS)?
4. Welche drei Felder braucht ihr wirklich?

Testet die URL erst im Browser oder mit `curl`, bevor ihr sie in den Code schreibt.

→ **Fertig, wenn** die vier Antworten in der README stehen und ihr die Rohdaten einmal gesehen habt.

A2 fetch einbauen

Holt die Daten mit `async` / `await`, prüft `res.ok`, fangt Fehler mit `try` / `catch`.

```
const res = await fetch(url);
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const daten = await res.json();
```

→ **Fertig, wenn** echte Daten aus dem Netz in eurer Anzeige landen und ein falscher Pfad in der URL eine Fehlermeldung erzeugt statt einer leeren Seite.

A3 Drei Zustände sichtbar machen

Eure Anzeige kennt ab jetzt **vier** Fälle. Alle vier müssen etwas zeigen:

Fall	Was der Nutzer sieht
lädt	„Lade …" oder ein Platzhalter
Daten da	das Ergebnis
nichts gefunden	ein Hinweistext, keine leere Fläche
Fehler	eine verständliche Meldung, kein Stacktrace

Testet den Ladezustand mit DevTools → Network → Throttling „Slow 3G". Testet den Fehlerfall, indem ihr die URL absichtlich kaputt macht.

→ **Fertig, wenn** ihr alle vier Fälle **vorführen** könnt, ohne den Code zu ändern.

A4 Eine Bibliothek bewerten

Sucht euch eine Bibliothek, die euer Projekt gebrauchen könnte (Diagramm, Karte, Tabelle, Datum). Prüft auf npmjs.com und bundlephobia.com: Downloads pro Woche, letzte Version, Größe, Lizenz.

Entscheidet – und begründet die Entscheidung in der README. **„Nein, brauchen wir nicht" ist eine vollwertige Antwort**, wenn die Begründung steht.

→ **Fertig, wenn** in der README steht, welche Bibliothek geprüft wurde, mit welchen Zahlen, und wie ihr entschieden habt.

A5 Klausurvorbereitung

Auf der Folie „Neun Fragen, ein Semester" stehen die Fragen, an denen ihr merkt, ob ihr den Kurs verstanden habt. Beantwortet sie **schriftlich in eigenen Worten**, für euch.

→ **Fertig, wenn** ihr bei jeder Frage ohne Nachschlagen drei bis fünf Sätze schreiben könnt. Die Lücken, die dabei auffallen, sind eure Lernliste.

Teil B · Bonus (+1 Punkt)

Schont die API und die Nerven eurer Nutzer:

- **Debounce:** nicht bei jedem Tastendruck laden, sondern 300 ms nach dem letzten
- **oder Caching:** dieselbe Anfrage innerhalb einer Minute aus `localStorage` beantworten

In die README: wie viele Anfragen ihr vorher und nachher auslöst.

Teil C · Verständnisfragen

1. Warum wirft `fetch` bei einem 404 keinen Fehler – und was macht ihr stattdessen?
2. Erklärt CORS in drei Sätzen: Wer blockiert, wen schützt das, und was hilft?
3. Was heißt „asynchron" – und was ginge schief, wenn `fetch` synchron wäre?

So arbeitest du hier mit KI

Modelle schreiben `fetch`-Code im Schlaf – und vergessen dabei fast immer den Lade- und den Fehlerzustand. Das ist eure Prüfaufgabe:

> *„Hier ist meine `fetch`-Funktion. Prüfe sie gegen diese vier Fälle: lädt, Daten da, leeres Ergebnis, Fehler. Sag mir, welcher fehlt – aber schreib den Code nicht neu."*

Für die API-Auswahl:

> *„Ich brauche eine öffentliche API für . Nenne mir drei, jeweils mit: Basis-URL, ob ein Key nötig ist, Rate-Limit und ob CORS erlaubt ist. Gib mir die Links zur Doku, damit ich es nachprüfen kann."*

Und dann prüft ihr nach. Rate-Limits und CORS-Angaben sind genau die Details, bei denen Modelle gern etwas erfinden, das plausibel klingt.